Continuation

Control.Monad.Trans.Cont: https://hackage.haskell.org/package/transformers-0.6.1.0/docs/Control-Monad-Trans-Cont.html
Control.Monad.Cont.Class: https://hackage.haskell.org/package/mtl-2.3/docs/Control-Monad-Cont-Class.html

```
newtype ContT r m a = ContT { runContT :: (a -> m r) -> m r }
type Cont r a = ContT r Identity a

instance MonadTrans (ContT r)
instance Functor (ContT r m)
instance Applicative (ContT r m)
instance Monad (ContT r m)
instance MonadFail m => MonadFail (ContT r m)

cont :: ((a -> r) -> r) -> Cont r a
runCont :: Cont r a -> (a -> r) -> r
evalCont :: Cont r r -> r
mapCont :: (r -> r) -> Cont r a -> Cont r a
withCont :: ((b -> r) -> (a -> r)) -> Cont r a -> Cont r b
reset :: Cont r r -> Cont r' r
shift :: ((a -> r) -> Cont r r) -> Cont r a

resetT :: (Monad m) => ContT r m r -> ContT r' m r
shiftT :: Monad m => ((a -> m r) -> ContT r m r) -> ContT r m a
evalContT :: (Monad m) => ContT r m r -> m r
mapContT :: (m r -> m r) -> ContT r m a -> ContT r m a
withContT :: ((b -> m r) -> (a -> m r)) -> ContT r m a -> ContT r m
b
callCC :: ((a -> ContT r m b) -> ContT r m a) -> ContT r m a
```

Continuation
Wiki - continuation: https://en.wikipedia.org/wiki/Continuation
Computation with callbacks
e.g.

```
-- direct style
id :: a -> a
id x = x

-- CPS style
idCPS :: a -> (a -> r) -> r
id x next = next x                --  next x = pass to next
computation

-- with Cont transform
idCPS :: a -> Cont r a
idCps x = cont (\next -> next x)
```

The function in CPS style is the **suspended computation**.
The return function is the **continuation**.
e.g.

```
idCPS :: a -> Cont r a
idCps x = cont (\next -> next x)    next :: a -> r = continuation
idCPS 1                             idCps 1 = suspended computation
i.e. suspended until continuation is passed
idCPS 1 print                       compute
```

```
---- Cont
Cont r a                        -- a=result of this computation
                                -- r=final output after all
computations
cont :: ((a -> r) -> r) -> Cont r a
cont (\k -> k 10)           -- Cont r Int
cont (\k -> k "Hello")      -- Cont r String

fn :: Cont r Int
fn = do
  x1 <- return 0         -- return next(0)
  x2 <- return (f x1)    -- return next(f(0))
  x3 <- return (g x2)    -- return next(g(f(0)))
  return x3              -- final return g(f(0))

-- e.g. computation reuse
fn :: Cont r Int                            -- f(f(f(2))) where
f(x)=10x
fn = do
x <- Cont (\k -> k $ k $ k 2)               -- reuse computation 3
times
return (x * 10)
evalCont fn                                 -- 10*10*10*2 = 2000
```

```
-- first
x = 2
cb 2 = x * 10 = 20

-- second
x = cb 2 = 20
cb $ cb 2 = x * 10 = 200

-- third
x = cb $ cb 2 = 200
cb $ cb $ cb 1 = x * 10 = 2000

--> result = 2000
```

```
-- e.g. short circuiting
fn :: Cont r Int
fn = do
  cont (\_ -> -1)     -- throw -1 i.e. bypasses all returns
  return 1            -- Never reaches here.
```

```
runCont fn (+10)        -- -1 i.e. +10 is never called
```

```
-- callCC: continue with early exit
--
-- Wiki: https://wiki.haskell.org/Continuation
--
--  ContT r m a               result monad
--   a                        result value
--  next0 :: a -> m r         base continuation
--  next  :: a -> ConT r m b  continue immediately
--                            i.e. `next 1` calls `next0 1`
immediately
--                            rest of chain ignored by ContT r m b
--
callCC :: ((a -> ContT r m b) -> ContT r m a) -> ContT r m a
```

```
-- implementation
callCC :: forall a r m b. ((a -> ContT r m b) -> ContT r m a) ->
ContT r m a

callCC f = ContT $ \next0 ->               -- next0 :: a -> m r  =
continuation
  let next :: a -> ContT r m b
      next a = ContT $ \_ -> next0 a

   in runContT (f next) next0              -- next0 is called
either by
                                          -- a) next called in f,
OR
                                          -- b) next is never
called in f, so next0 is called
```

```
x <- callCC (\next -> do
  when (...) $ next 1     -- call `next0 1` here and exit straight
to x
                          -- Rest is ignored
  ...
  return 10               -- call `next0 10` here and exit to x
)
print x                   -- 1 or 10 depending on exit


-- same as normal Cont if no early exit
callCC (\k -> k 1)
callCC (\_ -> return 1)      -- same
```

```
cont (\k -> k 1)              -- same
```

```
Early exit with/without callCC
cont (\_ -> ...) is also early exit, but this bypasses ALL
subsequent continuations
i.e. like throwing an error
callCC only exits early within the current computation.

e.g.
-- without callCC
isEvenSC :: Cont String String
isEvenSC n = do
  when (even n) $ cont (\_ -> "even")    -- bypass all
continuations. i.e. throw "even"
  return "odd"

-- with callCC
isEven :: Int -> Cont r String
isEven n = callCC (\cb -> do
  when (even n) $ cb "even"             -- early exit this
computation and continue.
  return "odd")

c = ("result:"++)             -- final continuation
runCont (isEven 0) c          -- result:even
runCont (isEven 1) c          -- result:odd
runCont (isEvenSC 0) c        -- even i.e. continuation c is
bypassed
runCont (isEvenSC 1) c        -- result:odd
```

```
-- delimited continuation: recursion in continuations
--
--   reset =
--   shift =
reset :: Cont r r -> Cont r' r
shift :: ((a -> r) -> Cont r r) -> Cont r a
fn = reset $ do                             -- f(f(f(2))) where
f(x)=10x
  x <- shift (\k -> k $ k $ k 2)            -- x=2 on first cb, x=20
on second cb, x=200 on last cb
  return $ x * 10
evalCont fn                                  -- 2000
```

```
-- same
fn = reset $ do
  x <- cont (\k -> k $ k $ k 2)
  return $ x * 10
```

```
runCont :: Cont r a -> (a -> r) -> r
evalCont :: Cont r r -> r
mapCont :: (r -> r) -> Cont r a -> Cont r a
withCont :: ((b -> r) -> (a -> r)) -> Cont r a -> Cont r b
reset :: Cont r r -> Cont r' r
shift :: ((a -> r) -> Cont r r) -> Cont r a

---- ContT
--
resetT :: (Monad m) => ContT r m r -> ContT r' m r
shiftT :: Monad m => ((a -> m r) -> ContT r m r) -> ContT r m a
evalContT :: (Monad m) => ContT r m r -> m r
mapContT :: (m r -> m r) -> ContT r m a -> ContT r m a
withContT :: ((b -> m r) -> (a -> m r)) -> ContT r m a -> ContT r m
b
```